

G05IART0.02 - Inteligência Artificial**Trabalho 01 - Busca Não Informada, Busca Informada e Busca Local****Semestre:** 2026/2 | **Valor:** 20,0 pontos**Entregas:** Parte 1 - 08/10/2026 | Parte 2 e final - 15/10/2026**Modalidade:** individual ou em dupla

1 Objetivos

O trabalho tem como objetivo implementar, comparar e analisar diferentes estratégias de busca estudadas na disciplina. A atividade está dividida em duas partes complementares:

Parte 1: busca **não informada e informada** aplicada à navegação em um ambiente ponderado;

Parte 2: busca **local** aplicada ao problema das 8 Rainhas.

A avaliação não considera apenas se o programa encontra uma resposta. O estudante ou a dupla deve demonstrar domínio sobre representação de estados, estruturas de dados, função de custo, heurísticas, critérios de expansão, limitações dos algoritmos e interpretação dos resultados experimentais.

2 Parte 1 - Busca não informada e informada em ambiente ponderado

Um agente deve deslocar-se em um ambiente retangular representado por uma grade. O objetivo é encontrar um caminho entre a posição inicial e o objetivo, considerando obstáculos e diferentes custos de travessia.

2.1 Formato do mapa

O programa deve ler o mapa de um arquivo texto retangular, sem dimensões fixas e sem posições de início/objetivo codificadas diretamente no programa.

Símbolo	Significado	Custo de entrada
S	estado inicial	-
G	objetivo	1
.	terreno normal	1
R	terreno irregular	3
M	terreno de alto custo	6
#	obstáculo	intransponível

O custo de uma ação é o custo da **célula de destino**. O custo total de uma solução é a soma dos custos de todas as células acessadas após sair de S. São permitidos apenas movimentos em quatro direções.

2.2 Algoritmos obrigatórios

Devem ser implementados em Python pelo estudante ou pela dupla:

- **buscas não informadas:** Busca em Largura (BFS), Busca em Profundidade (DFS) e Busca de Custo Uniforme (UCS);
- **buscas informadas:** Busca Gulosa pelo Melhor Primeiro e A*.

Não é permitido utilizar bibliotecas que implementem diretamente busca, menor caminho ou *pathfinding*, como rotinas equivalentes do NetworkX, SciPy, AIMA ou bibliotecas específicas de navegação. Bibliotecas como `numpy`, `pandas` e `matplotlib` podem ser utilizadas para tratamento de resultados e visualização, mas não para substituir os algoritmos solicitados.

2.3 Ordem de sucessores e desempates

Para tornar os resultados reproduzíveis, use obrigatoriamente a ordem lógica de sucessores:

Norte → Leste → Sul → Oeste

Para DFS, a pilha deve ser manipulada de forma que essa seja também a ordem efetiva de expansão. Estados de BFS e DFS devem ser marcados como descobertos quando inseridos na fronteira.

Nas filas de prioridade, utilize os seguintes critérios:

- UCS: $(g(n), \text{ordem de inserção})$;
- Gulosa: $(h(n), \text{ordem de inserção})$;
- A*: $(f(n), h(n), \text{ordem de inserção})$, com $f(n) = g(n) + h(n)$.

Em UCS e A*, se um caminho de menor custo para um estado ainda relevante for encontrado, o custo e o predecessor devem ser atualizados. Entradas obsoletas da fila de prioridade devem ser ignoradas e não contam como expansão.

2.4 Heurísticas

Implemente as duas funções abaixo:

$$h_M(n) = |x_n - x_G| + |y_n - y_G|$$

$$h_E(n) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2}$$

A comparação principal deve usar Manhattan. A Busca Gulosa e o A* também devem ser executados com a heurística Euclidiana para discussão do efeito da função heurística.

2.5 Mapa público de teste

O arquivo `data/mapa_teste.txt` acompanha o código-base:

```
S..MMMMM.....G
.#.#####.###...
.#.....#....#...
.#####.###.###...
.....#.#....#...
####.#.#####...
.....#...R.R....
.....
```

O programa será também executado pelo professor em **mapa(s) não disponibilizado(s) previamente**. Portanto, não serão aceitas implementações que dependam das dimensões, do desenho ou do caminho específico do mapa público.

2.6 Métricas obrigatórias

Para cada execução, registre:

- solução encontrada ou falha;
- número de ações da solução;
- custo total da solução;
- **nós gerados**: quantidade de inserções válidas na fronteira, incluindo o estado inicial; uma reinserção por melhora de custo em UCS/A* conta como nova geração;
- **nós expandidos**: estados retirados da fronteira para os quais a função sucessora é executada. Se o objetivo for reconhecido ao ser retirado da fronteira, ele não é contado como expandido;
- pico de estados distintos na fronteira;
- pico de estados armazenados: quantidade máxima de estados distintos presentes na união entre fronteira e explorados, isto é, $|fronteira \cup explorados|$;
- tempo de execução, como métrica secundária.

Para o tempo, execute cada configuração pelo menos 30 vezes e apresente a mediana, excluindo leitura do arquivo e geração de gráficos. O tempo não será usado para comparar computadores diferentes.

2.7 Tabela mínima de resultados

Alg.	Heur.	Passos	Custo	Ger.	Exp.	Pico F.	Pico Mem.
BFS	-						
DFS	-						
UCS	-						
Gulosa	Manhattan						
A*	Manhattan						
Gulosa	Euclidiana						
A*	Euclidiana						

A entrega final deve incluir pelo menos uma representação visual de caminho produzido pelos algoritmos no mapa público, em formato textual ou gráfico, identificando início, objetivo e percurso.

2.8 Testes mínimos obrigatórios

Além do mapa público, a implementação deve conter testes automatizados ou casos de teste reproduzíveis que contemplem, no mínimo:

- início adjacente ao objetivo;
- mapa sem solução;
- mapa com ciclos;
- existência de rotas distintas com o mesmo número de ações;
- existência de uma rota curta e cara e outra mais longa e barata.

Os testes não substituem a execução no mapa público nem os casos utilizados pelo professor durante a correção.

3 Parte 2 - Busca local: 8 Rainhas com Hill Climbing

O segundo problema consiste em posicionar oito rainhas em um tabuleiro 8×8 de forma que nenhuma rainha ataque outra.

3.1 Representação e função de avaliação

Use um vetor de tamanho 8 no qual o índice representa a coluna e o valor representa a linha da rainha. Exemplo:

```
board = [3, 6, 2, 7, 1, 4, 0, 5]
```

Há exatamente uma rainha por coluna. A função de avaliação deve ser:

$$h(s) = \text{número de pares não ordenados de rainhas em conflito.}$$

Logo, $h(s) = 0$ representa uma solução.

A vizinhança é composta por todos os movimentos de uma rainha para outra linha de sua própria coluna. A ordem de geração deve ser coluna 0 a 7 e, em cada coluna, linha 0 a 7, ignorando a posição atual. Cada estado possui 56 vizinhos.

3.2 Algoritmos obrigatórios

Implemente:

1. **Hill Climbing básico:** escolher o primeiro vizinho com o menor valor de h entre os melhores encontrados. O movimento só é realizado se houver melhora estrita. Não são permitidos movimentos laterais na versão obrigatória;
2. **Random-Restart Hill Climbing:** executar repetidamente o Hill Climbing básico a partir de novos estados aleatórios até encontrar solução ou alcançar o limite de 50 reinícios.

A geração inicial deve utilizar `random.Random(seed)` e uma chamada `randrange(8)` para cada uma das oito colunas. No Random Restart, reutilize o mesmo objeto gerador ao criar os estados iniciais sucessivos.

Para padronizar as métricas, uma **iteração** corresponde a um movimento de melhora efetivamente aceito. Em **estados avaliados**, conte o estado inicial e cada vizinho cujo valor de h seja calculado; avaliações repetidas em iterações distintas contam novamente. No Random Restart, iterações e estados avaliados devem ser acumulados entre todas as tentativas. Um **reinício** é uma nova inicialização após a primeira tentativa; portanto, o limite de 50 reinícios permite no máximo 51 tentativas.

3.3 Experimento obrigatório

Execute ambos os métodos com as sementes inteiras de 0 a 99. Apresente pelo menos:

- taxa de sucesso;
- média e mediana de iterações;
- média de estados avaliados;
- para Random Restart, média e máximo de reinícios necessários;
- análise detalhada de pelo menos uma execução do Hill Climbing que tenha parado sem solução, indicando o estado final e o valor de h .

4 Questões que a discussão deve responder

O relatório pode organizar a discussão livremente, mas deve responder de forma explícita às questões abaixo:

1. BFS encontrou o menor número de ações? Esse caminho também foi o de menor custo?
2. Por que BFS e UCS podem retornar soluções diferentes em um ambiente ponderado?
3. Como a ordem de sucessores influencia o comportamento da DFS?
4. Por que a Busca Gulosa pode expandir poucos estados e ainda produzir uma solução de custo elevado?
5. A* e UCS encontraram soluções de mesmo custo? Compare a quantidade de expansões.
6. Qual foi o efeito da troca de Manhattan por Euclidiana em Gulosa e A*?
7. As heurísticas utilizadas são admissíveis neste problema? Justifique considerando os movimentos e o menor custo de passo.
8. Por que Hill Climbing e Busca Gulosa não são equivalentes, embora ambos utilizem uma função heurística/de avaliação?
9. Explique uma falha do Hill Climbing em termos da paisagem de busca: mínimo local, platô ou outra situação observada.
10. Por que os reinícios aleatórios aumentam a chance de sucesso, mas não eliminam as limitações teóricas do Hill Climbing?
11. Compare as propriedades de completude e optimalidade de BFS, DFS, UCS, Busca Gulosa e A* nas condições adotadas neste trabalho. Relacione a análise teórica aos resultados observados.

5 Entrega e reprodutibilidade

O trabalho terá duas entregas, ambas realizadas pelo **SIGAA**. Além dos arquivos submetidos no SIGAA, o desenvolvimento deve ser mantido em um **repositório Git acessível ao professor**. A versão considerada em cada etapa será identificada por uma tag específica, de modo que alterações posteriores não modifiquem a versão originalmente entregue.

- **08/10/2026 - Entrega da Parte 1:** devem ser submetidos no SIGAA um arquivo **.zip** contendo a versão executável da Parte 1 e o **README.md** atualizado. O repositório deve conter as buscas não informadas (BFS, DFS e UCS), as buscas informadas (Busca Gulosa e A*), as duas heurísticas solicitadas, execução no mapa público de teste, testes mínimos e a tabela preliminar de resultados da Parte 1. O **README.md** deve informar o endereço do repositório e a tag **entrega-parte1**. **Não é necessário entregar o relatório final nesta data;**
- **15/10/2026 - Parte 2 e entrega final:** devem ser submetidos no SIGAA o relatório final em PDF e um arquivo **.zip** contendo o projeto completo. O repositório deve incluir a implementação de Hill Climbing e Random-Restart Hill Climbing para o problema das 8 Rainhas, juntamente com todo o conteúdo da Parte 1, experimentos finais, testes e documentação. O **README.md** e o relatório devem informar o endereço do repositório e a tag **entrega-final**.

O repositório deverá permanecer **privado até o encerramento da correção**, com acesso concedido ao professor. A versão avaliada será aquela apontada pela tag correspondente a cada entrega. Commits realizados posteriormente poderão permanecer no repositório, mas não substituirão a versão marcada pela tag. O histórico de commits poderá ser utilizado como elemento auxiliar na

verificação da autoria e da contribuição dos integrantes, sem exigência de quantidade mínima de commits.

Na entrega final, o material deve conter:

- repositório Git completo, acessível ao professor, com a tag **entrega-final**;
- arquivo **.zip** submetido no SIGAA contendo uma cópia da versão final do projeto;
- **README.md** com o endereço do repositório, identificação da tag da entrega e comandos de instalação e execução;
- **requirements.txt**, se houver dependências externas;
- script que reproduza as tabelas e resultados utilizados no relatório;
- testes básicos;
- **IA_USAGE.md** preenchido;
- em trabalhos em dupla, identificação objetiva da contribuição de cada integrante no relatório;
- relatório em PDF elaborado a partir do **modelo SBC fornecido** em **modelo_relatorio/**, com no máximo **6 páginas de conteúdo**, incluindo título, resumo e *abstract*. Não deve ser utilizada capa separada. As referências bibliográficas podem ultrapassar esse limite.

Os comandos de execução e a organização do código-base estão descritos no **README.md** do pacote fornecido. O código-base contém apenas a infraestrutura inicial; as funções correspondentes aos algoritmos e às heurísticas permanecem deliberadamente incompletas e devem ser implementadas pelo estudante ou pela dupla.

O relatório deve privilegiar metodologia, resultados e interpretação. O modelo SBC fornecido contém a estrutura recomendada, tabelas-base e orientações para a discussão.

6 Política de uso de IA e autoria

Ferramentas de IA podem ser utilizadas como apoio para revisão textual, esclarecimento conceitual, interpretação de mensagens de erro e discussão/criação de casos de teste, desde que o resultado seja compreendido, verificado e declarado.

É vedado utilizar IA ou código externo para fornecer a implementação pronta dos algoritmos solicitados e apresentá-la como produção do estudante ou da dupla. Também não é aceitável entregar uma análise de resultados gerada automaticamente sem revisão crítica dos autores.

Todo uso deve ser registrado em **IA_USAGE.md**, com ferramenta, finalidade, partes afetadas e forma de verificação. Recursos externos consultados devem ser citados. O professor poderá solicitar a qualquer integrante uma demonstração ou explicação breve da implementação, incluindo estruturas de dados, tratamento de repetição de estados, cálculo de custos, heurísticas e métricas.

7 Critérios de avaliação

Critério	Valor
Implementação de BFS e DFS	2,4
Implementação de UCS	1,6
Implementação de Busca Gulosa	1,6
Implementação de A*	2,4
Hill Climbing e Random Restart	3,0
Métricas, testes e metodologia experimental	3,0
Análise crítica e discussão dos resultados	3,0
Qualidade do código, organização e reprodutibilidade	2,0
Conformidade da entrega, autoria e declaração de IA	1,0
Total	20,0